

ubot: Master Setup and Work Log

What this is. This document records everything done to the ubot workspace between 29 September and 1 October 2026. It has three uses:

- rebuilding the whole setup from an empty machine (sections 1–4);
- running the benchmark (section 5);
- knowing what was changed and why (section 6).

[PROTOCOL.md](#) is the research design (what is measured and how it is analysed). This document is how to reproduce the environment and the work. The two are kept in sync.

Repository: [Unipod-Robotics/uni-bot](#) (GitHub). Branch `feat/nav-benchmark`, based on `imu-bno085-ekf`. Commits are listed in section 7.

1. Reference machine

The setup was built and tested on this machine. Other machines should match the bold items.

Item	Version
OS	Ubuntu 24.04 LTS (noble)
ROS 2	Jazzy (rc1py 7.1.11)
Gazebo	Harmonic (gz-harmonic 1.0.0, gz-sim 8, gz-physics 7 with DART)
gz_ros2_control	1.2.17
Nav2 / nav2_bringup	1.3.11
slam_toolbox	2.8.4 (a lifecycle node on Jazzy)
robot_localization	3.8.3
GPU	NVIDIA GeForce MX570 A, driver 580.159.03 (headless Gazebo rendering uses NVIDIA EGL)
CPU / RAM	12 cores / 31 GB
Python analysis	venv <code>~/uni-bot/.venv-bench</code> : evo 1.31.1, statsmodels 0.15.0, pandas 3.0.3, markdown 3.11

2. Fresh-machine setup

2.1 System packages

```
# ROS 2 Jazzy desktop and Gazebo Harmonic (ros-jazzy-ros-gz) are assumed installed.
sudo apt install ros-jazzy-nav2-bringup ros-jazzy-slam-toolbox ros-jazzy-robot-localization \
  ros-jazzy-gz-ros2-control ros-jazzy-ros2-controllers ros-jazzy-twist-stamper \
  ros-jazzy-depthimage-to-laserscan ros-jazzy-roslaunch2-storage-mcap ros-jazzy-xacro \
  python3-vcstool python3-scipy python3-psutil python3-venv
# Needed by the real-robot bringup and ubot_mono_nav (not installed on the reference machine
yet):
sudo apt install ros-jazzy-tf-transformations python3-smbus
```

2.2 Workspace

```
git clone git@github.com:Unipod-Robotics/uni-bot.git ~/uni-bot    # the repo root IS the
workspace
```

```
cd ~/uni-bot && git checkout feat/nav-benchmark
vcs import --skip-existing src < src/ubot/ubot.repos # third-party drivers (git-ignored)
src/ubot/ubot_worlds/scripts/fetch_models.sh # AWS world models, pinned
commit
```

- Cloning over HTTPS needs credentials; SSH was used for pushes.
- `fetch_models.sh` downloads ~140 MB from `zp78-ship-it/turtlebot-maze` at commit `30424d5` into `src/ubot/ubot_worlds/models_external/`, which is git-ignored.

2.3 Build

```
conda deactivate # an active conda env makes CMake pick the wrong Python
source /opt/ros/jazzy/setup.bash
cd ~/uni-bot
colcon build --symlink-install --cmake-args -DBUILD_TESTING=OFF
source install/setup.bash
```

- `-DBUILD_TESTING=OFF` is required: `topic_based_ros2_control`'s tests need `ros_testing`, which is not installed.
- `ubot_worlds` and `ubot_description` install **copies** of their data files. Rebuild them after editing a world, mission, map or URDF. `bench run` refuses to start if `ubot_worlds` is stale.

2.4 Analysis environment

```
python3 -m venv --system-site-packages ~/uni-bot/.venv-bench # keeps rclpy importable
~/uni-bot/.venv-bench/bin/pip install -r src/ubot/ubot_bench/requirements-bench.txt markdown
```

2.5 Headless rendering

Gazebo sensors need a GPU context even headless. The orchestrator sets this automatically; for manual runs:

```
export __EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/10_nvidia.json
```

3. Check that the setup works

Check	Command	Expected
Interactive sim	<code>ros2 launch ubot_bringup sim.launch.py (add world:=arena_5x5, sensor_profile:=lidar_ld06 as needed)</code>	Gazebo shows the small house, the robot spawns, RViz shows /scan and the IMU axes on the robot
Teleop	<code>ros2 run teleop_twist_keyboard teleop_twist_keyboard</code>	Drives in Gazebo with no sideways creep
Worlds render	<code>ros2 launch ubot_worlds gz_world.launch.py world:=bookstore condition:=glass</code>	World loads with no Ogre crash
Smoke experiment	<code>ros2 run ubot_bench bench run smoke</code>	4 trials (REF + OAKD, arena, both phases) ok
Analysis	<code>~/uni-bot/.venv-bench/bin/python -m ubot_bench.analysis.report smoke</code>	<code>bench_results/smoke/analysis/</code> holds CSVs, figures and tables

4. Regenerating the committed data (only if worlds change)

Ground-truth maps and missions are committed. They are regenerated only when a world or the generation code changes, and always in this order:

```
ros2 run ubot_bench gt_map --world arena_5x5          # ~1 min
ros2 run ubot_bench gt_map --world small_house        # ~20 min under load
ros2 run ubot_bench gt_map --world bookstore          # ~15 min
ros2 run ubot_bench gt_map --world small_warehouse --lattice 0.75
for w in arena_5x5 small_house bookstore small_warehouse; do ros2 run ubot_bench make_missions
--world $w; done
colcon build --packages-select ubot_worlds            # install the new missions and maps
```

- Inspect `ubot_worlds/missions/<world>.png`. Goals and the route must be inside the building.
- The AWS world xacros are generated by `ubot_worlds/scripts/import_aws_world.py <src> <out> <name>` from the pinned `tb_worlds/worlds/*.sdf.xacro`. Re-run it only if the import logic changes.

5. Running the benchmark

Experiment	Purpose	Trial s	Command
pilot	Validate the pipeline; power estimate	24	<code>ros2 run ubot_bench bench run pilot</code>
sim_core	RQ1/2/4: 4 stacks × 4 worlds × 20 seeds	640	<code>ros2 run ubot_bench bench run sim_core --parallel 3</code>
sim_core_tight	SLAM 0.1 m / 0.1 rad ablation, paired with sim_core	640	<code>ros2 run ubot_bench bench run sim_core_tight --parallel 3</code>
sim_degradation	RQ3: glass, walkers, degradation, low light	~1,120	<code>ros2 run ubot_bench bench run sim_degradation --parallel 3</code>
sim_to_real	RQ5: sim half of the real comparison	480	<code>ros2 run ubot_bench bench run sim_to_real --parallel 3</code>

Analysis:

```
~/uni-bot/.venv-bench/bin/python -m ubot_bench.analysis.report <experiment>
~/uni-bot/.venv-bench/bin/python -m ubot_bench.analysis.compare sim_core sim_core_tight
ros2 run ubot_bench bench list <experiment> -v          # progress per trial
```

Rules for campaigns

1. Run from a **clean, committed** tree. Each experiment records the commit and a dirty flag (`experiment.yaml`, `code_version`).
2. Close everything heavy first: interactive Gazebo, RViz, OBS. During the pilot the machine sat at load ≈ 50 on 12 cores (real-time factor ≈ 0.3). That is slow and it caused timeouts.
3. Run long jobs as user services so they survive closing terminals and sessions: `bash systemd-run --user --collect --unit=bench-sim-core --working-directory=$HOME/uni-bot \ bash -c 'source /opt/ros/jazzy/setup.bash; source install/setup.bash; \ ros2 run ubot_bench bench run sim_core --parallel 3 > bench_results/sim_core.log 2>&1' systemctl --user status bench-sim-core # running? systemctl --user stop bench-sim-core # stop the orchestrator If you stop it, kill the trial processes too (each trial is tagged GZ_PARTITION=bench-<trial>): bash for d in /proc/[0-9]*; do { tr '\0' '\n' < $d/environ; } 2>/dev/null | grep -q '^GZ_PARTITION=bench-' && kill -9 ${d#/proc/}; done`

4. Results are resumable. Re-running an experiment skips finished trials and retries infrastructure failures once. Task failures are data and are never retried.

6. Work log (chronological)

6.1 29 Sep 2026: workspace moved to the latest branch

- **Problem.** `~/uni-bot/src/ubot` was an old, non-git copy.
- **Done.**
 - `~/uni-bot` became a checkout of Unipod-Robotics/uni-bot, branch `imu-bno085-ekf` (the newest; it contains main, mono-nav, inspection-robot-description and hardware-interface).
 - The old `src/ubot` and `src/MS200_ros` moved to `~/uni-bot-backup-2026-09-29/`. 17 files there were edited but never pushed (old box-model URDF, nav2/SLAM tuning).
 - Missing drivers imported: `bno08x_ros2_driver`, `topic_based_ros2_control`.
 - Clean build of 11 packages, with `-DBUILD_TESTING=OFF` (section 2.3).

6.2 29 Sep 2026: "the robot slides in Gazebo"; IMU axes stuck at the origin

- **Finding.** Headless ground-truth tests found 0.0 cm sideways drift in Gazebo, so the slide was in the estimated pose shown in RViz.
- **IMU axes at the map origin.** The RViz Imu display listened to `/bno055/imu`, which nothing publishes. Now every consumer uses `/imu`: the sim bridge remap was removed in `sim.launch.py` and `sim_mono.launch.py`, and `slam.rviz` and `sim_ekf.yaml` were updated.
- **Sim EKF.** It fused absolute x (not y) and absolute yaw from the wheels; wheel yaw under-reads turns by $\sim 4\%$. It is now velocities only (wheel vx and yaw rate, IMU yaw rate), the same as `real_ekf.yaml`.
- **Sim gyro.** It used MPU-9250 noise, with a random bias up to $\sim 0.3^\circ/\text{s}$. It now uses the BNO085 value measured in the E2 still test ($\sigma 0.0019 \text{ rad/s}$) with no static bias.
- **Result.** End-of-run heading error dropped from 3.2° to 0.1° on a 2 m / 90° / 2 m drive.
- **Commits** `cc1502c` and `4940ed9` (`slam_toolbox scan_topic: /scan`), pushed to `imu-bno085-ekf`.
- **Report.** `~/uni-bot/src/ubot_sim_imu_ekf_report.pdf`.

6.3 30 Sep 2026: decisions for the paper

Decision	Choice
Venue	IROS 2027 (deadline 1 Mar 2027). ICRA 2027 (16 Sep 2026) had passed
Real-robot ground truth	Overhead camera + fiducials
Hardware	Budget LiDARs (MS200, LD06) and OAK-D Lite. No reference-grade LiDAR , so REF is simulation-only and sim-to-real predictivity becomes RQ5
Scope of the build	Protocol plus full simulation benchmark; real-robot parts specified, built later
1 Oct: SLAM thresholds	Keep <code>slam_toolbox</code> defaults (0.5 m / 0.5 rad) and add the tight 0.1 m / 0.1 rad run (<code>sim_core_tight</code>)
1 Oct: RPLIDAR A1	Not used; removed from profiles, experiments, figures and protocol

6.4 30 Sep to 1 Oct 2026: benchmark built (commits `dc8cfbd`, `0fd141e`, and section 7)

Worlds (`ubot_worlds`)

- The worlds on the suggested list are Gazebo Classic. The house, bookstore and warehouse were taken from their community Harmonic ports, pinned in `turtlebot-maze/tb_worlds`; the hospital has no port. `import_aws_world.py` regenerates them:
 - our shared systems and lighting replace the originals;
 - the roof is dropped;
 - furniture is forced static. Many AWS models are non-static mesh bodies, which had put the house at 0.03× real time;
 - ceiling lights become switchable for the low-light condition.
- `arena_5x5` is new, and it is the physical arena's build spec.
- Conditions are xacro args: glass panes (collision, invisible to LiDAR/depth via `visibility_flags`), walkers (actors), low light. Their placements live in `missions/<world>.yaml`.
- **Physics step 2 ms** in every world (the bookstore ran at 0.35× real time at 1 ms).
- The worlds do **not** load the Sensors system: the robot URDF does, and loading it twice crashed Ogre.

Sensors (`ubot_description`)

- `config/sensor_profiles.yaml` holds the datasheet values for REF (UST-10LX), MS200, LD06 and OAK-D Lite. The MS200 manual gives 4,500 points/s (450 beams at 10 Hz); the "40,000 points/s" figure found online is the MS200k.
- The URDF builds the LiDAR from the profile (`sensor_profile:=`). `bench:=true` publishes a clean `/scan_raw` for `scan_model` to add datasheet noise; otherwise Gazebo adds constant noise on `/scan`.
- The bench bumper is a contact sensor on all chassis collisions. Gazebo publishes it on the scoped default topic, not the URDF `<topic>`.
- Wheel friction direction set to the axle (`fdir1 0 0 1`, `mu1` sideways 0.5, `mu2` rolling 1.0).
- In sim the rear wheels are driven explicitly (command interfaces; the `diff_drive` drives all 4) instead of mimic joints, which matches the ESP32 mirroring on the real robot.

Harness (`ubot_bench`)

- `scan_model` (noise), `gt_publisher` (ground truth and collision episodes), `gt_map` (ground-truth maps), `make_missions`, `mapping_runner`, `mission_runner`, `bench` (orchestrator) and `analysis/` (metrics, stats, figures, report, compare).
- `slam_toolbox` is started through its own launch file because it is a lifecycle node on Jazzy.
- OAK-D depth → scan reuses `ubot_mono_nav/depth_to_scan`, with its measured mount (TF off, because `camera_depth_frame` is not an optical frame). A centre-row `depthimage_to_laserscan` would see the floor at ~2.1 m because of the 4.09° tilt.

Findings and fixes during the build

#	Finding	Fix
1	Headless Gazebo used software rendering	NVIDIA EGL vendor variable (section 2.5)

#	Finding	Fix
2	Bookstore storefront is open at the 0.36 m scan plane (sill below), so goals were generated outside the building	Three-slice ground-truth scanner (0.08 / 0.22 / 0.36 m): scan-plane map plus traversability map
3	Warehouse bounds were guessed from a skewed, drift-built map	Bounds set just inside the measured walls ($x \pm 6.85$, $y \pm 10.33$ m)
4	In-place turns pivot about the front axle (base moves 0.25 m per 180°)	Not a bug. Coulomb friction makes the more-loaded axle the pivot (CoM $x = +0.013$ m); a 16 mm rearward CoM flips it to the rear axle. Collision shape, friction direction, wheel order, mimic and torque limits had no effect. Kept, documented, and measured on the real robot; position-only success added
5	Mapping route zig-zag (47°/m) made the follower crawl	Route straightening
6	The ground-truth follower hit a box	Route clearance 0.30 / 0.35 m, lookahead 0.30 m, <code>stuck</code> detection
7	Regenerated missions were not installed, so trials mixed old and new routes	Stale-install guard in <code>bench_run</code>
8	Lifecycle activation timed out under load, so a trial hung	Bounded Nav2 startup wait; retried once
9	Nav2 behaviour-tree server timeout of 20 ms aborted goals under load	1000 ms (plus <code>wait_for_service_timeout</code> 5000 ms)
1 0	Background jobs died with the editor session	Long jobs run as user systemd services (section 5)

Each experiment that ran before a fix was set aside in `bench_results/_invalid_*` or `_pilot_arena_nav_lowtimeout` and re-run. Those results are not used.

Figures palette. The figures use categorical slots 1–4 of the reference palette for REF, MS200, LD06 and OAK-D, in that order (`#2a78d6`, `#eb6834`, `#1baf7a`, `#eda100`). The validator passes (worst adjacent CVD ΔE 9.1, normal-vision ΔE 22.9). Two colours are below 3:1 contrast, so every figure also carries visible stack labels.

6.5 Pilot status (1 Oct 2026, 02:30)

- The pilot runs as a user service, `ubot-pilot2`. The follow-up service `ubot-pilot-followup` waits for it, re-runs the set-aside arena navigation trials, then writes `bench_results/pilot/analysis/`. Its log is `bench_results/pilot_followup.log`.
- Arena results so far:** SLAM ATE is REF 7.2 cm vs MS200 7.5 cm, and map F1 is 1.00 for both. The navigation numbers are being re-run after fix #9.
- House:** the mapping pair completed; navigation is running.

7. Commits

Commit	Branch	Summary
<code>6ab133d</code>	<code>imu-bno085-ekf (base)</code>	BNO085 covariances from the E2 still test (pre-existing)
<code>cc1502c</code>	<code>imu-bno085-ekf (pushed)</code>	Sim EKF velocities-only, BNO085 gyro noise, <code>/imu</code> everywhere

Commit	Branch	Summary
4940ed9	imu-bno085-ekf (pushed)	slam_toolbox scan_topic: /scan
dc8cfbd	feat/nav-benchmark	Benchmark: worlds, sensor models, harness, protocol
0fd141e	feat/nav-benchmark	Pilot fixes, all ground truth and missions, protocol v1.2
(this commit)	feat/nav-benchmark	Tight-SLAM ablation, RPLIDAR A1 removed, Sources section, this log

feat/nav-benchmark is **not pushed**. Push it with `git push git@github.com:Unipod-Robotics/uni-bot.git feat/nav-benchmark`.

8. Where things are

Path	What
src/ubot/ubot_bench/docs/PROTOCOL.md / .pdf	Research protocol, including all sources
src/ubot/ubot_bench/docs/REPRODUCE.md / .pdf	This document
src/ubot/ubot_bench/	Harness code, experiments (experiments/*.yaml), Nav2/SLAM configs
src/ubot/ubot_worlds/	Worlds, missions (+ PNG previews), ground-truth maps, model fetch script
src/ubot/ubot_description/config/sensor_profiles.yaml	Sensor models (datasheet values)
~/uni-bot/bench_results/<experiment>/	Trial outputs and analysis/ (git-ignored)
~/uni-bot/.venv-bench/	Analysis venv (git-ignored)
~/uni-bot-backup-2026-09-29/	Pre-migration copy with 17 unpushed local edits
~/uni-bot/src/ubot_sim_imu_ekf_report.pdf	IMU/EKF investigation report (section 6.2)

9. Open items

- Not built yet for the real robot: the overhead-camera ground-truth node (AprilTag, ≥ 20 Hz, TUM output), the INA219 power logging, and scripted mapping drives (PROTOCOL.md section 7).
- Sensor prices are approximate. Replace them with dated quotes.
- The BNO085's residual gyro bias has not been measured, so the sim gyro has zero static bias.
- Measure the real robot's in-place pivot point with the ground-truth rig.
- Install `ros-jazzy-tf-transformations` and `python3-smbus` (needed by the real-robot bringup).
- `ubot_bringup/launch/real_robot.launch.py` still carries a commented-out RPLIDAR A1 block from before this work; it is unused.